Code Cave 扫描与 .text 节空隙利用

- **.text 节为什么会有空隙**

(一) PE 节区的对齐规则

PE 文件在磁盘上时，节区按照 FileAlignment（通常是 0x200，即 512 字节）对齐。加载到内存时，节区按照 SectionAlignment（通常是 0x1000，即 4KB）对齐。

举例：假设某个 DLL 的 .text 节：

- 磁盘上的实际代码大小：0x1A00 字节
- 磁盘对齐后占用：0x1A00（向上取整到 0x200 的倍数，正好是 0x1A00）
- 内存对齐后占用：0x2000（向上取整到 0x1000 的倍数）

结果：内存里的 .text 节从 0x1A00 到 0x2000 之间有 0x600（1536 字节）的空隙。

- **空隙里填的是什么**

链接器在填充这些空隙时，通常会填入以下两种字节：

- 0x00：未初始化区域，常见于编译器的默认行为。
- 0xCC：调试断点指令（int 3），微软的 MSVC 编译器在 Debug 模式下喜欢用这个填充。

物理意义：这些字节在 CPU 眼里是合法的可执行指令（0x00 是 add [rax], al，0xCC 是断点），但程序永远不会执行到这里（因为代码段逻辑不会跳到填充区）。所以它是天然的可执行空地。

- **代码示例（见文件夹）**

(一) 明文 API 实现

关键物理动作解释：

步骤	物理意义
ReadProcessMemory(moduleBase, localDos, 0x40, ...)	只读 64 字节的 DOS 头，不用把整个 PE 头搬到本地
section.VirtualAddress	节区在内存中的偏移量（RVA）
alignedSize - virtualSize	空隙大小
sectionStart + virtualSize	空隙起始地址
没有 VirtualProtectEx	因为 .text 节本身已经是 PAGE_EXECUTE_READ，它的 padding 天然可执行

(二) 无 IAT 版本（复用前些课的.h 文件）

- **两版对比总结**

对比项	第一版	第二版
IAT 里有敏感函数名	有	无
字符串里有明文	有	无
静态扫描检测率	高	低
代码复杂度	简单	中等（需初始化函数指针）

注：本方法因 CFG 现已不适用，属 Code Cave 注入，非模块踩踏